



Variables de Condición



Una **variable de condición** es un mecanismo de sincronización asociado **siempre** a un cerrojo. Permite que un hilo espere hasta que se cumpla una condición lógica, y que otros hilos puedan notificar cambios sobre esa condición.

Explicación

Las variables de condición aparecen originalmente en el **modelo teórico de monitores**, donde cada condición tiene su **propia cola de espera**.

Aunque los lenguajes reales no obligan a encapsularlas en monitores, **su uso SOLO ES CORRECTO si existe exclusión mutua alrededor de ellas**.

Una variable de condición permite:

- suspender un hilo cuando una condición NO se cumple,
- liberando temporalmente el cerrojo,
- y reanudarlo cuando otro hilo actualiza el estado y envía una señal.

Por eso **no sustituyen a un cerrojo**, sino que lo complementan.

Gracias a ella podemos tratar de forma distinta la exclusión mutua, ya que podemos hacerlo con un cerrojo por ejemplo, y la sincronización, ya que podemos hacerlo con variables de condición.

Cada variable de condición tiene una cola asociada EN TEORÍA. Esta cola NO ES LA MISMA que la de exclusión mutua del cerrojo, son 2 colas separadas.

En la implementación cambia la cosa debido al diseño de los lenguajes de programación.

Además, la relación que tengan estas colas con la del cerrojo asociado vendrá dada por la política de señalización que se haya definido (para saber más, ver los

apuntes de señalización).

Implementación

En implementación SIEMPRE se usa `while` en vez de `if`, porque no se garantizan prioridades ni condiciones verdaderas al despertar.

¿Por qué usamos while?

En la teoría (Hoare), quien hace `signal` **cede el monitor inmediatamente** al hilo despertado y se garantiza que la condición será verdadera cuando despierte.

En la práctica (Java, C++, Python):

- No se garantiza prioridad.
- Puede despertar un hilo **cualquiera**.
- Al reanudarse, la condición puede **seguir siendo falsa**.
- También pueden ocurrir *spurious wakeups* (despertares espurios).

Por eso, las implementaciones usan SIEMPRE:

```
while (!condición) wait();
```

y **nunca**:

```
if (!condición) wait(); // INCORRECTO en implementación real
```

Este "while" es lo que se llama **cláusula de guarda**.

Java API Standart

En esta versión de la API **NO TENEMOS VARIABLES DE CONDICIÓN**.

La única cola de espera que podemos tener es el wait-set asociado a cada objeto. Por tanto, todas las condiciones en un mismo objeto comparten esa única cola.

Para manipular el wait-set hacemos uso de los métodos `wait()`, `notify()` y `notifyAll()`, definidos en la clase `Object` y utilizados dentro de un bloque sincronizado (`synchronized`)

```
synchronized (lock) {
    while (!condicion) {
        lock.wait();
    }
    // sección crítica después de esperar
}
```

Java API Alto Nivel

Aquí sí existen **variables de condición reales con su respectiva cola por cada variable de condición**.

Cada `ReentrantLock` puede crear múltiples condiciones:

```
Lock lock = new ReentrantLock();
Condition noVacio = lock.newCondition();
Condition noLleno = lock.newCondition();
```

Métodos (su cambio de nombre se debe a que siguen heredando de Object, por lo que serán un objeto que tenga un cerrojo interno y unos métodos finales `wait()`, `notify()`, y `notifyAll()`):

- `await()` → equivalente a `wait()`
- `signal()` → equivalente a `notify()`
- `signalAll()` → equivalente a `notifyAll()`

No obstante, estas colas NO SON ACCESIBLES POR EL PROGRAMADOR DIRECTAMENTE (tú no puedes llamar a la cola y manipularla como haces con un array). Su manipulación se hace a través de los métodos descritos arriba.

C++

C++11 permite múltiples variables de condición asociadas a un mutex compartido:

- `wait(unique_lock)` → libera el mutex y se bloquea
- `notify_one()` → despierta uno
- `notify_all()` → despierta todos

Ejemplo típico:

```
std::mutex m;  
std::condition_variable cond;  
bool listo = false;  
  
void esperar() {  
    std::unique_lock<std::mutex> lk(m);  
    cond.wait(lk, []{ return listo; }); // cláusula de guarda integrada  
}
```